

คู่มือสรุปเน้นออกสอบวิชา Embedded System Design (Master Printable PDF)

รวมชุดคำสั่ง AVR Assembly ครบทุกคำสั่ง และคำอธิบายโค้ดตัวอย่างบรรทัดต่อบรรทัด (Line-by-Line Code Explanations)

● PART 1: สรุปเน้นสอบตอนเช้า (ปิดตำรา - CLOSED BOOK)

1.1 K-Map 4x4, Full Adder & CPU Pipeline

- **DeMorgan:** $(A \cdot B)' = A' + B'$, Minterm (m_i) ใน SOP เอาต์พุต=1
- **Full Adder & Overflow:** $Sum = A \oplus B \oplus C_{in}$, $V = C_{in}(MSB) \oplus C_{out}(MSB)$
- **Flip-Flops:** D ($Q(t+1)=D$), JK ($Q(t+1)=J \cdot Q' + K' \cdot Q$), T ($Q(t+1)=T \oplus Q$)
- **Control Unit:** Hardwired FSM (เร็ว ยืดหยุ่นน้อย เหมาะ RISC) vs Microprogrammed ROM Store (ช้ากว่า แก้ไขง่าย เหมาะ CISC)

PART 2: สรุปเน้นสอบตอนบ่าย (เปิดตำรา - OPEN BOOK & ALL ASM)

2.1 คลังชุดคำสั่งภาษา AVR Assembly ครบทุกคำสั่ง (All 40+ Instructions Table)

หมวดคำสั่ง	คำสั่ง (Opcode)	รูปแบบ (Syntax)	การทำงาน (Operation) & คำอธิบาย
ย้ายข้อมูล	LDI / MOV / MOVW	LDI Rd, K	โหลดค่าคงที่ K ลง Rd (SR_16-R_31\$) / `MOVW` ก๊อปปี้ค่า 16 บิต
	LDS / STS	LDS Rd, k / STS k, Rr	อ่าน/เขียนข้อมูลตรงกับแอดเดรส k ใน Data SRAM
	IN / OUT	IN Rd, A / OUT A, Rr	อ่าน/เขียนข้อมูลพอร์ต I/O Register A (เช่น PORTB, PINC)
	PUSH / POP / LPM	PUSH Rr / LPM Rd, Z	เก็บ/ดึง Stack / อ่านข้อมูล 1 ไบต์จาก Flash ROM ชี้โดย Z Pointer
คำนวณ ALU	ADD / ADC	ADD Rd, Rr	บวก Rd = Rd + Rr (`ADC` บวก Carry C เพิ่มอีก 1)
	SUB / SUBI / SBC	SUBI Rd, K	ลบด้วยค่าคงที่ K หรือลบรวมตัวยืม Carry C
	INC / DEC / NEG	INC Rd / NEG Rd	เพิ่ม/ลดค่าทีละ 1 / หาค่า 2's Complement เปลี่ยนเครื่องหมาย
	MUL / MULS / CPI	MUL Rd, Rr	คูณ 8 บิต ผลลัพธ์ 16 บิตในคู่ `R1:R0` / `CPI` เปรียบเทียบเซต SREG
ลอจิก & บิต	ANDI / ORI / EOR	ANDI Rd, K	ลอจิก AND Mask บิต / ORI เซตบิต 1 / EOR XOR บิต
	SBI / CBI	SBI A, b / CBI A, b	Set Bit 1 หรือ Clear Bit 0 ของพอร์ต I/O Register A ขา b
	SBIS / SBIC	SBIS A, b	ข้าม 1 คำสั่งถ้าบิตพอร์ต I/O b เป็น 1 (`SBIS`) หรือ 0 (`SBIC`)
กระโดดเงื่อนไข	RJMP / CALL / RET	CALL Subroutine	กระโดดไร้เงื่อนไข / เรียกฟังก์ชันย่อย และดึงกลับด้วย `RET`
	BREQ / BRNE	BRNE label	กระโดดเมื่อ Z=1 (เท่ากัน) หรือ Z=0 (ไม่เท่ากัน)
	BRLO / BRSH	BRLO label	กระโดดเมื่อ C=1 (น้อยกว่า) หรือ C=0 (มากกว่า/เท่ากับ)
	SEI / CLI / RETI	SEI / RETI	เปิด Global Interrupt (I=1) / ดึง PC คืนจาก Interrupt ISR

2.2 อธิบายโค้ดตัวอย่างบรรทัดต่อบรรทัด (Line-by-Line Code Explanations)

📁 1. การตั้งค่า Stack Pointer (SPH:SPL = RAMEND)

```
.INCLUDE "M328PDEF.INC"
.ORG 0x0000
LDI R16, HIGH(RAMEND)
OUT SPH, R16
LDI R16, LOW(RAMEND)
OUT SPL, R16
```

💡 คำอธิบายบรรทัดต่อบรรทัด:

- .INCLUDE & .ORG: ดึงไฟล์นิยามแอดเดรสชิป และตั้งจุดเริ่มต้นที่ Flash 0x0000
- LDI R16, HIGH(RAMEND) & OUT SPH, R16: ดึง 8 บิตสูงของ RAMEND (0x08) ขึ้นแอดเดรสสูงสุด SRAM ลง SPH
- LDI R16, LOW(RAMEND) & OUT SPL, R16: ดึง 8 บิตต่ำของ RAMEND (0xFF) ลง SPL ให้ Stack พร้อมใช้งาน

📁 2. อัลกอริทึมแปลง Hex เป็น BCD 2 หลัก (ลบซ้ำด้วย 10)

```
LDI R16, 0x45      ; ค่าอินพุต 0x45 = 69 ทศนิยม
CLR R17            ; R17 เก็บหลักสิบ (Tens)

BCD_LOOP:
    CPI R16, 10
    BRLO BCD_DONE
    SUBI R16, 10    ; ลบออกทีละ 10
    INC R17         ; เพิ่มหลักสิบ
    RJMP BCD_LOOP

BCD_DONE:
    SWAP R17
    OR R17, R16     ; รวมหลักสิบและหลักหน่วย (0x69)
    OUT PORTB, R17
```

💡 คำอธิบายบรรทัดต่อบรรทัด:

- LDI R16, 0x45 & CLR R17: กำหนดค่าอินพุตเริ่มต้น 69 และล้างค่าหลักสิบ R17 เป็น 0
- CPI R16, 10 & BRLO BCD_DONE: เช็คค่าเหลือถึง 10 ไหม ถ้าน้อยกว่า 10 จบloop (เศษใน R16 คือหลักหน่วย)
- SUBI R16, 10 & INC R17: ลบออกทีละ 10 และเพิ่มค่านับหลักสิบ R17 ขึ้นทีละ 1
- SWAP R17 & OR R17, R16: ย้ายหลักสิบขึ้นไป 4 บิตสูง (SWAP) แล้ว OR รวมหลักหน่วย ได้ BCD 0x69

📁 3. โค้ดแล็บจริง: AssemblyBlink1.asm (ควบคุมไฟกะพริบพร้อมกัน 2 พอร์ต)

```

;=====
; Title: AssemblyBlink1.asm
; Created: 11/22/2013
; Author: khatathapswatdipisal
; Description: Blink all LEDs on PORTB and PORTD simultaneously
;=====

.nolist
.include "m328pdef.inc"
.list

; Start at main after reset
rjmp main

main:
    ldi r16, 0xFF          ; Load 0xFF into R16 (all bits set to 1)
    out DDRB, r16          ; Configure Port B pins as Outputs
    out DDRD, r16          ; Configure Port D pins as Outputs
    ldi r16, 0x00          ; Clear R16 to 0x00

loop:
    com r16                ; Complement R16 (toggle bits between 0x00 and 0xFF)
    out PORTB, r16         ; Write R16 to Port B
    out PORTD, r16         ; Write R16 to Port D
    call MY_DELAY          ; Delay for simulation timing
    rjmp loop              ; Loop indefinitely

; Delay Subroutine
MY_DELAY:
    ldi r20, 8
L1:
    ldi r21, 200
L2:
    ldi r22, 250
L3:
    dec r22                ; Decrement R22
    brne L3                ; Loop if R22 != 0
    dec r21                ; Decrement R21
    brne L2                ; Loop if R21 != 0
    dec r20                ; Decrement R20
    brne L1                ; Loop if R20 != 0
    ret                    ; Return from subroutine

```

💡 คำอธิบายบรรทัดต่อบรรทัด:

- out DDRB/DDRD, r16: ตั้งพิน PORTB และ PORTD ทั้งหมดเป็นเอาต์พุต
- com r16 & out PORTB/PORTD, r16: กลับบิต R16 สลับ 0x00 → 0xFF ทำให้ไฟกะพริบติดดับพร้อมกัน
- call MY_DELAY: เรียกฟังก์ชันย่อยหน่วงเวลา 3 ลูปซ้อนกัน (R20, R21, R22)